The Latent Monitor Audit: What We Built, How It Works, and What It Found

A technical report on the inference-only study of continuous-thought reasoners

Muhammad Jon Raza
Project repository: `0x0shephard/latent-reasoning`
Research ledger §108–§136, October 2026

Abstract

Some language models solve math word problems without writing their steps. Instead they produce six hidden vectors, called "thoughts", and then the answer. Several groups want to watch these models by training a small classifier (a "probe") to read the thoughts. This report explains what we did to test whether that works. We built an audit that needs no training and asks three separate questions about each thought: does it *carry* a value into the answer, does the model *need* it, and when a probe reads a value from it, does the model's behaviour *confirm* that reading. We ran the audit on eight released models, five datasets and 39 preregistered Kaggle runs that reproduce byte for byte. The short version of what we found: a probe can read a thought well even when the model never uses it, and the model can use a thought it does not strictly need. Below 3B, a thought carries a value only on problems with three or more steps. One or two thoughts do the carrying and pass the value to the next position's memory. The last thought is readable and never used. Probes also read facts about the question from every thought, so a probe can look accurate while reading nothing the model computed. The thoughts predict wrong answers no better than the model's own confidence. The one practical gain is not in the thoughts: send the quarter of problems the model is least sure about to its own written solution, and accuracy rises by 3.7, 7.5 and 10.2 points at 1B, 3B and 8B. We also report the first result of a new experiment: a one-vector write into the right thought steers 21% of correct answers to a value an attacker chose, the same write into the unused thought sets off a probe on 100% of cases while changing nothing, and, against our prediction, the thought after the written one keeps the old value, so a detector there catches the attack at 0.92 AUROC.

1 Why we did this

The models. CODI (Shen et al., 2025), SIM-CoT (Wei et al., 2026) and Coconut (Hao et al., 2024) solve grade-school math problems without writing out the steps. After reading the question, the model runs six extra forward passes. Each one produces a vector. These six vectors are the "thoughts". Each thought is handed to the next one through a small projector, and each thought also leaves behind keys and values in the attention cache, which we will simply call its **memory**. Every later step can look back at that memory. Then the model writes the answer. Nobody can read the thoughts directly.

The idea we tested. A natural fix is to train a probe to read the thoughts. Ramjee et al. (2026) trained a model with a hidden "bad intention", showed a probe on the early thoughts detects it 89% of the time, and recommended monitoring the early thoughts. But that result, like every probe result

Preprint. Under review.

we could find, only shows that the information is *present* in the vector. It does not show the model *uses* it. A probe can read a note the model threw away.

What made us look. We had spent months on these models for another reason, and had built tools to swap, delete and overwrite single thoughts inside the official checkpoints. Two facts did not fit together. In our experiments, swapping the vector of thought 1 or 3 between two problems changed about a third of the answers. SCIT (Ding et al., 2026), on the same checkpoint, swapped the *memory* instead and found that is what carries the answer, while the vector "is weak". Both can be true only if a thought acts through two different routes, and a probe reads only one of them. One more thing made this urgent: the released CODI code throws away the sixth thought's output vector. A probe on z_6 reads something that goes nowhere.

The question. When a thought changes the answer, which route does the work? Does the thought carry a specific value or just disturb things? Does the model need it? And does a probe's reading track any of this? All of these can be answered from the released weights, with no training, in about one GPU-hour per model.

2 Models and data

Table 1: The checkpoints. "Ours" is our GSM8K-test accuracy on 1,319 problems. (The CODI-1B number fixes a figure other papers quote: 51.9% is a side experiment in the CODI paper; its main result is 55.6%.)

Name	Base	Source	Paper	Ours
CODI GPT-2	GPT-2 (124M)	`zen-E/CODI-gpt2`	43.7	43.6
CODI-1B	LLaMA-3.2-1B-Instruct	`zen-E/CODI-llama3.2-1b-Instruct`	55.6	55.5
SIM-CoT-1B	LLaMA-3.2-1B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-1B`	56.1	55.7
SIM-CoT-3B	LLaMA-3.2-3B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-3B`	62.3	61.9
SIM-CoT-8B	LLaMA-3.1-8B-Instruct	`internlm/SIM_COT-LLaMA3-CODI-8B`	64.1	62.5
Coconut	GPT-2 (124M)	`UdeSA/coconut-0.2B-gsm8k-aug` (retrain)	34.1	34.6
Coconut on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-coconut`	97.6	97.6
CODI on ProsQA	LLaMA-3.2-1B-Instruct	`connordilgren/llama32-1b-prosqa-codi`	99.0	99.0

Every experiment needs problems whose solution is written as a chain of equations, like `«8+1=9» «9*80=720» «720-700=20»`. We use GSM8k-Aug training problems (the set CODI was trained on; 2,048 rows, plus 14,336 more for long chains), the original GSM8K test set (1,206 usable rows, 854 with three or more steps), the original GSM8K train set (6,998 rows), ASDiv (1,490 rows, only 86 with three or more steps), and SVAMP as a one-step control. "Short" means one or two steps; "long" means three or more. Everything ran on Kaggle T4 GPUs, from two accounts.

3 The instruments, explained

Every measurement below is an intervention: we change something inside the model while it runs, and watch what the answer does. Before each run we check the plumbing: feeding a thought its own recorded vector, or its own memory, must reproduce the original output exactly. It always did.

3.1 Two routes out of every thought

A thought at position s can affect the answer in two ways.

- Its **vector** (we also say "state") is handed to thought $s{+}1$. Think of it as the front door.
- Its **memory** (keys and values) sits in the cache, and every later step can look at it. Think of it as the back door.

Our hooks can edit either one alone: replace the vector before it is handed on, or overwrite the memory after it is written.

One fact follows from the architecture, and we checked it exactly on all eight models: swapping all six vectors *and* all six memories gives the same result as swapping the memories alone. The last vector is thrown away, so a vector can only act through the memory its successors write. This is why a probe on vectors is reading a half-way object.

Worked example: the two-route audit on CODI-GPT2 (§108–§109)

We took 512 training problems and paired each with an unrelated one. At each of the six positions we swapped in the other problem's vector only, or its memory only, or both, and counted how often the first token of the answer changed. Result: the vector matters at the odd positions (31% and 37% of answers change), the memory matters at the even positions (42%, 24%, 28%), and within the memory it is the values at layers 8–9 that do it. The sixth thought changes 0% of answers through its vector and 3.9% through its memory. Replacing the whole set of six memories changes 75% of answers, but only 4.7% of them become the *donor's* answer: the model mixes the stored values with its own question when it answers.

3.2 Twins: does a thought carry a value, or only disturb?

If you swap in a thought from an unrelated problem, answers break. But breaking is not the same as carrying. To test carrying we build **twins**. Take a problem, change one number in the question, and recompute the chain, so the twin's answer is the right answer to the changed question. For example, "12 cucumbers" becomes "13 cucumbers" and `«12*5=60» «54+60=114»` becomes `«13*5=65» «54+65=119»`. We only change a number that appears once in the question and once in the chain, is not itself a result, and keeps all results whole and non-negative.

- **Only solved pairs count.** We keep a pair only if the model gets both twins right. Otherwise "the answer followed the swapped value" means nothing.
- **The swap.** The two twins sit next to each other in one batch. We replace one site in the first twin (one thought's vector, one position's memory, or all of them) with the second twin's, and read the first answer token.
- **The outcome.** If the token becomes the twin's answer we call it *target*. If it stays the original answer, *retain*. Anything else, *other*. The share of *target* is the **transport** rate: how often the answer followed the number. Confidence intervals come from a bootstrap over pairs.
- **Short and long.** The *standard* set changes the first eligible number and is mostly short chains. The *long* set keeps only chains of three or more steps and changes the number that enters the chain last. Each has up to 512 pairs.
- **Where the value sits.** We can also apply the model's own output layer to a thought's vector and read off a few numbers. If the original decodes its number at some thought and the twin decodes the twin's number at the same thought, we say the value is *located* there.

3.3 Masking: does the model need the thought?

Carrying is not needing. For each thought we try two removals: *mean* replaces its vector with the average vector over all problems, so nothing specific to this problem is passed on; *mask* hides its memory from every later step through the attention mask. (The second needs a decoding path that keeps the attention mask through every step. The released LLaMA code drops it; we fixed that.) We report how many accuracy points are lost, separately for short and long chains, with a bootstrap. Three checks were fixed in advance: short chains lose at most 5 points from any single removal (K1), the carrying thoughts cost at least 10 points on long chains (K2), the last thought costs at most 2 points (K3).

3.4 Probes: readable, and faithful?

A probe is a small model, trained after the fact, that predicts a label from a recorded thought vector. We use three readers: the model's own output layer (no training), a linear ridge probe, and a small two-layer network. The label is the chain's second-to-last intermediate result, the value the carrying thoughts hold. (The final answer itself cannot be read from any thought; probes score 3%.) Probes are trained on four fifths of the pairs and tested on the fifth they did not see, five times over, with both twins of a pair always on the same side. **Readability** is the test accuracy.

Faithfulness is what turns this into an audit. After we swap a thought, the model holds the twin's vector at that position. A probe sitting there would read the twin's value v' and raise an alarm: "the model is now working with v'". Faithfulness is the share of those alarms the model's behaviour

confirms, that is, how often the answer actually became the twin's:

$$\text{faithfulness}(s) = \frac{\#\{\text{rows where the probe reads } v' \text{ at } s \text{ and the answer became the twin's}\}}{\#\text{rows}}.$$

A thought can be very readable and never faithful. The same probes, with different labels (the chain's first operator; whether the problem has three or more steps), tell us whether a probe is reading the plan or just the question. We call these the **prompt-leakage** probes.

3.5 The card

One command runs everything above on a model and a dataset and writes one *card*. The thresholds are fixed. A *carrier* is a thought whose vector transports at least 0.15 on long chains. A *store* is a position whose memory transports at least 0.15 while its vector does not. On short chains nothing should transport above 0.10 (except position 0's memory, which holds the question). *Necessary* means at least 10 points lost on long chains. *Readable* means probe accuracy at least 0.30, and *readable-but-unfaithful* means readable with alarms confirmed at most 5% of the time. Five checks give a *regime*: `critical_path` if every carrier is necessary, `backup_route` if carriers exist but are not necessary, `no_transport` if there is no carrier. The card also records the GPU, library versions, the dataset hash and a fingerprint of every run's predictions, so two cards from two sessions can be compared bit for bit.

4 Findings on the CODI family

Each box gives the claim, how we got it, and the numbers. Section numbers point to the research ledger.

F1. Disturbing a thought is not the same as the thought carrying a value (§109, §111)

Swap thought 3 with an unrelated problem's and 37% of answers change. Swap it with the twin's, which differs in one number, and the answer follows that number only 10% of the time; 86–92% of the time it stays put. Thought 1: 31% vs 5%. Most of the "change" was just disruption. *How:* the same 512 problems, scored once with random donors and once with twins.

F2. Below 3B, a thought carries a value only on problems with three or more steps (§111–§113, §117)

On one- and two-step problems no single thought transports above 0.10. The model reads the answer straight off the question; the thoughts hold a readable trace that is not used. On three-step and longer problems, thought 3's vector moves the answer to the twin's in about a third of cases: 0.325 [0.293, 0.358] on GPT-2 and 0.339 [0.304, 0.375] on CODI-1B. *How:* the long twin set, built after we noticed the effect in small cells and wrote the prediction down first.

F3. One or two thoughts carry; they pass the value to the next position's memory; the last thought (below 3B) never matters (§109, §113, §117, §120)

The value goes from the carrier's vector into the *whole* memory of the next position (CODI-1B: vector_3 0.339, memory_4 0.260, vector_4 0.000, memory_3 0.000), not into the two-layer value block SCIT pointed at (0.047 on GPT-2). That block is what *breaks* answers, not what *carries* them. When the model's own output layer says which thought holds the value (CODI-1B, 60% of rows, almost always thought 3), that thought carries 0.529 and the other feeding thought 0.000: the thought that decodes the value is the one that carries it. The sixth thought decodes the value as well as thought 3 and transports 0.000 through both routes on 590 rows; removing it costs at most 1.9 points on every model up to 1B.

F4. The carrier is used but barely needed; the question copy is the most needed site (§120–§121, §124)

On CODI-1B the carrier moves the answer a third of the time, yet averaging it away costs only 2.7 points and hiding its memory costs 0.0. When a thought is removed the model simply recomputes from the question. We call this *sufficient but not necessary*, the `backup_route` regime, and it is the usual case on natural GSM8K questions. Meanwhile, hiding the memory of position 0, which holds a copy of the question written before any thought, costs 37.6 points on GPT-2 and 58.6 on SIM-CoT-1B. A probe that reads the thoughts and ignores position 0 misses the site the model leans on most.

Table 2: Readability / faithfulness per thought on long chains (linear probe; label = the second-to-last intermediate value). In every model the last thought is readable and faithful exactly 0% of the time.

thought	GPT-2	CODI-1B	SIM-CoT-1B
0	0.09 / 0.00	0.27 / 0.02	0.25 / 0.02
1	0.33 / **0.19**	0.15 / 0.01	0.62 / **0.21**
2	0.46 / 0.00	0.36 / 0.02	0.69 / 0.02
3	0.40 / **0.40**	0.79 / **0.36**	0.72 / **0.28**
4	0.54 / 0.00	0.84 / 0.05	0.74 / 0.03
5	0.42 / **0.00**	0.81 / **0.00**	0.57 / **0.00**

F5. Readable is not faithful; probes read the question; the readable-but-unused thought holds a stale copy (§120–§121, §124–§125, §133)

Table 2: on CODI-1B a probe reads thought 4 at 0.84 accuracy and its alarm is right 5% of the time; thought 5 reads at 0.81 and is right never. This did not depend on the probe: the linear and non-linear probes gave faithfulness within 0.05 of each other. We first wrote "the most readable thought is an unused one" and had to withdraw it, because the non-linear probe sometimes put the carrier on top. What survives on every card up to 3B is weaker but solid: there is always a thought that reads at 0.30 or better whose alarm is right at most 5% of the time, and the last thought is that thought in every model. Probes also read facts about the question from *every* thought: "does this problem have three or more steps" decodes at 0.76–0.95 from all six, position 0 and the dead last thought included. Finally, the readable-but-unused thought is not reading the question. On CODI-1B thought 5 reads the computed intermediate value at 0.47, *better* than the carrier (0.42). It holds a copy of a value that was already used and passed on. Readability outlives use.

F6. Which thoughts carry is set by training, and a training change moved it as we predicted (§115, §118–§119)

GPT-2 carries at thoughts 1 and 3 (stored at 2 and 4). CODI-1B carries at thought 3 only (stored at 4). SIM-CoT-1B, the same backbone trained with extra supervision on every step, carries at both 1 and 3 again (0.191/0.185 and 0.247/0.229, against CODI-1B's 0.011/0.011 and 0.339/0.260); we wrote that prediction before the run. SIM-CoT also turns position 0 into a copy of the question that transports 0.54 of short-chain twins on its own. One prediction failed: we expected step supervision to make the model's own output layer point to one thought; instead it pointed to several (unique location fell from 60% to 6%), because the output layer is not the decoder SIM-CoT trained.

5 Four uses of the map

5.1 A probe can read the question instead of the plan (§125)

Things fixed by the question alone (the first operator; whether there are three or more steps) decode above chance at every thought of every model, including the two thoughts that do nothing. So a probe evaluated on something the prompt already determines (a trigger word, a task type, a difficulty level)

can score well while reading nothing the model computed. For a backdoor test where the trigger is in the prompt, this is the confound. The fix is cheap: run the same probe on a prompt-only control at the same thought.

5.2 The thoughts do not catch errors better than confidence (§125, §131, §134)

For every GSM8K test problem we recorded the thoughts and the answer, read each thought with the model's own output layer (top value, its probability, entropy, agreement with the answer), and trained classifiers to predict "this answer is wrong": from the carrier thoughts only, the unused thoughts only, all thoughts, the raw vectors, and, as a baseline, the model's confidence in its first answer token (Table 3). Confidence wins on every model. The thoughts add at most 0.01 AUROC, even with 12,288 raw features. One useful thing did come out: on the 1B–8B models the error signal inside the thoughts sits exactly at the carriers the swaps had found (CODI-1B carriers 0.82 vs unused 0.60), which confirms the map from a task it was never fitted on.

Table 3: AUROC for "the answer is wrong" on the full GSM8K test set (1,319 problems).

Model	Wrong	Confidence	Carriers	Unused	All thoughts	Raw vectors
GPT-2	746	**0.889**	0.850	0.853	0.851	0.859
CODI-1B	587	**0.912**	0.821	0.596	0.827	0.835
SIM-CoT-1B	584	**0.913**	0.754	0.650	0.799	0.836
SIM-CoT-3B	502	**0.915**	0.793	0.688	0.807	0.844
SIM-CoT-8B	495	**0.930**	0.763	0.665	0.794	0.824

5.3 Writing the right value into the carrier (§126–§130)

If the carrier holds the intermediate value and passes it on, can we use that channel backwards and fix a wrong answer?

Worked example (a real CODI-1B case)

"The elevator in Jack's building supports a maximum load of 700 kg. An adult weighs on average 80 kg. If Jack rides with 8 other adults, by how much will they exceed the maximum load?" The correct chain is `«8+1=9» «9*80=720» «720-700=20»`, so the needed intermediate is $v^* = 720$. CODI-1B answers 40. Its output layer reads 640 at the carrier: it multiplied 8 by 80 and forgot Jack. We keep a bank of 2,048 training problems the model solved, and at five of them the carrier reads 720. We copy the most similar one's carrier vector into thought 3 while the model decodes, and read the new answer: 20. Fixed. With a donor holding a random *different* value, the answer stays 40.

Protocol. Bank: 2,048 training problems decoded once, vectors and memories saved, indexed by the value read at the carrier. Targets: wrong answers on three-step-or-longer test problems (up to 512), plus 128 correct ones to measure harm. Conditions: the nearest same-value donor, a random same-value donor, the *average* of all same-value vectors, a *random different-value* donor (the key control: if this fixed as many, the fix would be luck, not the value), the store's memory, vector plus memory, and the whole donor tail.

Table 4: Repair rate (%) on wrong answers whose carrier read the wrong value and that had a donor; harm (%) on correct answers. Value from the gold chain.

Model	n	Nearest	Rand. same	Average	Diff. value	Store mem.	Harm
GPT-2	357	15.1 [11.5, 19.0]	17.6	20.2	3.6	11.5	18.7
CODI-1B	266	22.6 [17.7, 27.8]	21.8	23.7	2.6	10.2	7.0
SIM-CoT-1B	273	15.4 [11.4, 19.8]	9.9	13.2	2.6	4.0	5.2
SIM-CoT-3B	239	14.2 [10.0, 18.8]	12.1	15.9	1.7	6.3	8.6
SIM-CoT-8B	232	14.7 [10.3, 19.4]	14.7	13.4	6.0	5.2	7.8

What we learned. The carrier can be written to: 14–23% of failures are fixed, against 2–4% for a different-value donor, with 5–9% harm at 1B and 3B. Two predictions were wrong in useful ways. The average of same-value vectors works as well as the best single one, so searching for the right

donor buys nothing. And adding the store's memory makes repair *worse*, because the donor's memory carries the donor's whole problem. A learned map from a number to a carrier vector (§128) lands in the same 20–24% band. Writing several thoughts at once (§129) is no better and harms more. When the needed value comes from the model's own written solution plus a calculator, repair still works on CODI-1B (18.8%), but the written solution's own answer is right on more of those same failures than the repair recovers.

Why the ceiling is low (§129–§130, §133). We sorted every wrong answer by what the model's own written solution does on that problem. *Plan wrong*, 48–68%: the written solution is wrong too. *Latent-only*, 19–53%: right when writing, wrong when thinking silently. *Upstream arithmetic*, 6–11%. *Final step*, 1–9%. Writes fix at most 2.6% of plan-wrong failures; they already fix a third of the latent-only ones they can reach. Latent-only failures are hard problems, not slips: another model of the same size gets them wrong 60% of the time too. Handing the model the correct plan at inference (§132) recovers 6–8% of plan-wrong failures, against 3–7% for a *wrong* plan, so that is noise, not help.

Net accuracy. On the 838 long-chain test problems the best gated write gains -0.1 / $+3.4$ / $+1.1$ / $+0.4$ / $+3.4$ points (GPT-2 / CODI-1B / SIM-CoT-1B / 3B / 8B), about two thirds of that on the full test set. Simply switching to the model's written solution when it disagrees gains $+5.6$ / $+5.3$ / $+3.2$ / $+9.7$ / $+21.3$. So the write shows the mechanism is real; it is not a method that beats reading the written solution.

5.4 Route on confidence: the cheap gain (§131, §133–§134)

Confidence is the best error detector we have, and the model's written mode gets many of the latent failures right. So we used confidence as a switch: think silently on every problem, and only write out the solution when the first-token probability is below a threshold. Sweeping the threshold gives Table 5. This needed no new GPU time for four of the five models; the data were already on disk.

Table 5: Confidence-gated routing: accuracy change (points) on the 838 long-chain GSM8K test problems when the least-confident fraction of problems is sent to the model's own written solution. 95% intervals at 25% and 50%.

Routed	GPT-2	CODI-1B	SIM-CoT-1B	3B	8B
10%	$+0.8$	$+0.8$	$+0.5$	$+2.7$	$+4.9$
25%	$+1.5$ [-0.2, $+3.2$]	$+3.7$ [$+1.9$, $+5.1$]	$+1.9$ [-0.3, $+4.3$]	$+7.5$ [$+5.1$, $+9.9$]	$+10.2$ [$+8.1$, $+12.6$]
50%	$+3.1$ [$+0.1$, $+5.7$]	$+7.6$ [$+4.5$, $+9.7$]	$+4.9$ [$+1.6$, $+7.8$]	$+11.1$ [$+7.5$, $+14.3$]	$+19.1$ [$+15.9$, $+22.0$]
100% (always written)	$+5.6$	$+5.3$	$+3.2$	$+9.7$	$+21.3$
Oracle (route exactly the wrong ones)	$+12.0$	$+11.5$	$+10.3$	$+17.2$	$+23.9$
Best gated thought write	$+1.0$	$+3.4$	$+1.1$	$+0.9$	$+3.4$

Three things to notice. At 1B and 3B, routing *half* the problems beats always writing (7.6 vs 5.3; 11.1 vs 9.7): the written solution is wrong on some problems the confident silent pass got right, and the gate keeps those away from it. Adding the six thought vectors to the gate makes it *worse* at 25% routed on three of four models; confidence is the whole gate. And the same confidence splits the thought write's wins from its losses: the write works best on confident wrong answers (73% vs 38% between top and bottom quartile on CODI-1B) and breaks loosely held correct ones (19–31% harm in the bottom quartile, 0–3% in the top). Below the threshold, switching wins; above it, doing nothing wins. The write comes out ahead in exactly one cell (CODI-1B, high confidence, $+1.2$ vs $+0.5$). **Route on confidence. Do not write into the thoughts.**

6 Contrast and scale

Bigger models (§130, §134). The two findings that defined the small models turn out to be small-model properties. At 3B the carriers move to positions 2 and 4 with the store at 3; position 4 also carries on short chains (so the chain-length gate is gone); the last thought's memory carries 0.15 and costs 8 points (so it is in use). At 8B there are two carrier pairs, one for short chains (4→5, transport 0.51) and one for long (2→3, 0.28); the question copy costs 5 points instead of 33–59; and there is no readable-but-unused thought left: readability and faithfulness now move together (correlation 0.71). What holds at every size: carriers are used but not needed, the work lives at one or two trained

positions, the thoughts add nothing to error detection, and the gap between silent and written mode grows (5, 10, 21 points).

Coconut is the opposite case (§127). A third-party retrain with the official recipe, audited with the same tools. Its pairing is (0,1), (2,3), (4,5), exactly as its two-thoughts-per-step training predicts, and it held to the third decimal. But its thoughts carry on one-step problems, every pair is needed (45–49 points), the last position is in use, and a transplanted thought drags in the donor's whole problem (repair 5.6% vs 3.1% control, 29% harm). Same six positions, same task, opposite profile. The CODI results come from CODI's training, not from thoughts in general.

ProsQA: nothing to monitor (§135). Coconut and CODI at 1B on a synthetic logic task (97.6% and 99.0%). Transport is 0.000 at every position and for the whole set; removing all six thoughts costs 0.000. And when we rewrite one premise so the other answer becomes correct, the models give the *old* answer 68–79% of the time: the high accuracy does not come from following the reasoning path. The audit's third profile, `no_transport`. There is nothing in those thoughts to watch and nothing to fix.

7 Preregistration and reproducibility

Every experiment went into the ledger with its predictions before it ran, and was scored after. Eleven predictions were partly or wholly wrong and are reported as such; the most useful wrong ones were that the 1B model's output layer would not point to one thought (it did, 60%), that step supervision would make it point to one (it did not), that the averaged vector would lose to the best single donor (it did not), that the store's memory would help repair (it hurt), and that the 3B model would carry at one thought (it carries at two). We also recorded two mistakes of our own: a benchmark number taken from the wrong table, and a check reported as passing without reading the committed card. Five reruns on two Kaggle accounts, days apart, reproduced their prediction fingerprints byte for byte. Code, cards, per-problem outcomes and the ledger are in the repository; every notebook pins the commit it runs.

8 What is new, against the literature

Checked claim by claim on 2026-10-12 (ledger §122) and for the cascade on 2026-10-17.

ThinkRouter must be read in full before submission. If it routes per problem on CODI-style models using first-token confidence, our routing contribution shrinks to the negative half (the thoughts add nothing) and the scale and oracle analysis.

9 New experiment: the targeted latent hijack (§136)

The repair work wrote the *right* value into a *failed* problem. An attacker would do the reverse: write a *wrong* value into a *solved* problem. This turns the whole story into an attack and a defence. If the carrier can be written to, if the later thoughts are copies, and if confidence rather than the thoughts carries the error signal, then: one vector should steer the answer to a chosen value; every probe downstream should agree with the attacker; a write at the unused thought should set off a probe while changing nothing; and only recomputing the answer should catch it.

Protocol. Victims: up to 512 correctly answered three-step-or-longer GSM8K test problems. For each, we fix the attacker's value v_{att} *before* any write: the bank value closest to v^* whose *implied answer* (the correct chain's last equation re-evaluated with v_{att}) is a whole, non-negative number other than the correct answer. Conditions: an own-value control (a different bank row that reads the victim's own v^*), the nearest attacker donor, a random same-value donor, the average, a *random-other* donor reading neither value (the control for "it is the value, not the disturbance"), the store's memory, the learned writer, and a donor written at the readable-but-unused *decoy* thought. Outcome per victim: *hijacked* (the implied answer), *broken* (some other wrong answer), *intact*. On the defender's side we record what every thought reads after the write, the model's confidence before and after, linear detectors on the vectors (trained on clean vs written, tested on victims they did not see), and a check against the model's own written solution.

Table 6: Novelty by finding. "Prior" is the closest earlier work; "ours" is what we did not find anywhere.

Finding	Closest prior	Ours
Disturbing $\neq$ carrying; the chain-length gate	Cywiński et al. (2025) on 2-step templates, naming more steps as open; Ding et al. (2026); Li et al. (2026)	Same pairs scored both ways, on natural data and official checkpoints; the gate itself with intervals
Two routes with the exact identity; the trained pairing	SCIT swaps values only; papers that recompute the cache from the residual	The split without recomputing; pairings (1,3), {3}, (1,3), (2,4), (2→3, 4→5), (0,1)(2,3)(4,5) across models
The question copy at position 0	—	Transports 0.54 on short chains; costs 59 points when hidden
Faithfulness per thought (alarm confirmation)	"Decodable is not causal" in principle (Open Problems, 2025; Decodability, 2026); Ramjee et al. (2026) use probes with no causal test	The quantity, measured per thought on latent reasoners; last thought readable 0.42–0.81, faithful 0.000
Used but not needed; need depends on training and size	Jin et al. (2026); Li et al. (2026) report parts separately	Swap and removal on the same pairs; CODI-1B transports 0.34 and costs 2.7
The stale copy	—	The unused thought reads the computed value better than the carrier
The last thought is unused	Partly prior (Cywiński et al., 2025)	Three checkpoints, both routes, with the probe consequence
Value-matched repair	Chang et al. (2026): geometric edits, no value-matched donor or controls	Natural failed problems, different-value control, harm, average $\approx$ best donor, memory route hurts, two value sources, the ceiling
Error detection fails vs confidence	Latent-reward classifiers (LTO, 2026)	The negative on latent reasoners plus the placement confirmation
Confidence-gated routing	FrugalGPT-style cascades; confidence-gated CoT (Lewis-Lim et al., 2510.21007); ThinkRouter (2602.11683)	Released fixed-thought models, no training; thoughts add nothing to the gate; routing half beats always writing at 1B/3B
Failure sorting; latent-only failures are hard problems	—	Stable across four models, computable at inference
Scale 1B→3B→8B; Coconut contrast; ProsQA third profile	Dilgren and Wiegreffe (2026) drop thoughts on logic tasks	The causal audit at 3B/8B; the rewritten-premise result on ProsQA

Predictions, written before the run. H1 hijacked ≥ 0.15 at CODI-1B and 8B. H2 hijacked $\geq 3\times$ the random-other donor's rate. H3 own-value control intact ≥ 0.90. H4 the decoy thought reads v_{att} on $\geq 50\%$ of victims while $\leq 5\%$ of answers change. H5 after a hijack the last thought reads v_{att} on $\geq 50\%$. H6 confidence AUROC ≤ 0.70, carrier detector ≤ 0.70, all-thought detector ≤ 0.80. H7 the written-solution check catches $\geq 80\%$ of hijacks at $\leq 40\%$ false alarms.

CODI-1B result (first run, 394 s on one T4). 416 correctly answered victims; an attacker value and a donor exist for 410. Table 7 gives the outcomes.

Table 7: The targeted hijack on CODI-1B: share of outcomes over 410 victims. The attacker's value is written at carrier thought 3 unless stated.

Condition	Hijacked	Broken	Intact
Own-value control (another row reading the victim's own v^*)	0.002	0.061	**0.937**
Nearest attacker donor	**0.210** [0.171, 0.249]	0.173	0.617
Random same-value donor	0.124	0.202	0.673
Average of same-value vectors	0.151	0.188	0.661
Random-other donor (neither value)	**0.015**	0.339	0.646
Store memory only / vector + store memory	0.032 / 0.068	0.120 / 0.154	0.849 / 0.778
Learned writer, absolute / residual	0.102 / 0.159	0.156 / 0.146	0.741 / 0.695
Decoy write at thought 5	0.000	0.000	**1.000**

In plain terms

What held. The attack works and it is aimed. One natural vector written into thought 3, with no text produced, moves a fifth of correct answers to the attacker's chosen value and breaks another sixth; a donor carrying some other value hijacks only 1.5% (H1, H2). Writing the victim's own value changes almost nothing (94% intact, H3). The decoy is total: the same kind of write at thought 5 makes a probe there read the attacker's value on 100% of 296 victims and changes the answer on none (H4). Checking against the model's own written solution flags every hijack at a 27% false-alarm rate, or 98% at 15% when comparing final answers (H7).

What did not hold, and why it matters more. We predicted the later thoughts would copy the attacker's value (H5). They did not. After a successful hijack, thought 4 still reads the *original* value on 53% of cases and the attacker's on only 13%; thought 5 reads the attacker's on 6%. So after the write, the carrier disagrees with its own store. That disagreement is visible: a linear detector on thought 4 alone tells hijacked from clean at 0.92 AUROC (0.95 for all written runs), all six thoughts together 0.92, while the written thought itself is nearly blind (0.70) and thoughts 0–2 are at chance. Confidence also drops, from 0.91 to 0.84 on hijacked runs (AUROC 0.78). The stale copy that made a probe *unfaithful* in Section 5 is, under attack, *tamper evidence*: the value the carrier handed on survives in the next positions, and a write that changes the carrier but not them leaves a seam. This is the first positive result for a thought-based monitor in the whole study, and it comes from checking that the thoughts agree with each other, not from reading any one of them. SIM-CoT-3B (two carriers) and 8B (no decoy thought; readability tracks faithfulness) are running now, to see whether the seam survives at scale.

10 Limitations, and what we would do next

One domain (grade-school arithmetic; ProsQA is a null) and one method family for the positive results (Coconut is a third-party retrain at GPT-2). ASDiv has few long chains. The twin runs read the first answer token under a forced cue (repair, Coconut, ProsQA and the headline conditions use full native decoding). The 8B ran in float16 on two T4s and scored 1.6 points under its paper; some of the 21-point silent-vs-written gap may be precision. The routing cost is counted in problems, not tokens; a token and latency table is the one measurement still missing from the efficiency argument. The most significant next step is to test the published safety monitor itself causally: replicate Ramjee et al. (2026) with their code, train the inert-trigger control their design lacks, and run the audit on the armed state.

References

D. Aswal, T. Palmeira Ferraz, Y. Zhou, and M. Peyrard. Observable patterns are not explanations: A causal-geometric analysis of latent reasoning models. *arXiv:2606.12689*, 2026.

S. Chang et al. Unlocking the black box of latent reasoning: An interpretability-guided approach to intervention. In *ACL*, 2026. arXiv:2606.01243.

B. Cywiński, B. Bussmann, A. Conmy, J. Engels, N. Nanda, and S. Rajamanoharan. Can we interpret latent reasoning using current mechanistic interpretability tools? *Alignment Forum*, December 2025.

J. Dilgren and S. Wiegreffe. Are latent reasoning models easily interpretable? *arXiv:2604.04902*, 2026.

Ding, Huang, and Yang. SCIT. In *EMNLP*, 2026. arXiv:2608.27265.

S. Hao et al. Training large language models to reason in a continuous latent space. *arXiv:2412.06769*, 2024.

Jin, Yang, and Wang. Final checkpoints are not enough: Analyzing latent reasoning faithfulness along training trajectories. *arXiv:2607.06648*, 2026.

Li et al. Dynamics within latent chain-of-thought: An empirical study of causal structure. In *ICML*, 2026. arXiv:2602.08783.

Latent thinking optimization: Your latent reasoning language model secretly encodes reward signals in its latent thoughts. In *ICLR*, 2026. arXiv:2509.26314.

Open problems in mechanistic interpretability. *arXiv:2501.16496*, 2025.

Decodability is not causality: Dissociating probe readouts from behavioral drivers via SAE decomposition. *arXiv:2609.18080*, 2026.

Ramjee et al. Ulterior motives: Detecting misaligned reasoning in continuous thought models. In *ICLR*, 2026. arXiv:2604.23460.

Z. Shen et al. CODI: Compressing chain-of-thought into continuous space via self-distillation. In *EMNLP*, 2025. arXiv:2502.21074.

X. Wei et al. SIM-CoT: Supervised implicit chain-of-thought. In *ICLR*, 2026. arXiv:2509.20317.